

The PEACE Protocol¹

A protocol for transferable encryption rights.

Logical Mechanism LLC²

March 30, 2026

¹This project was funded in Fund 14 of Project Catalyst.

²Contact: support@logicalmechanism.io

Contents

1	Abstract	1
2	Introduction	1
3	Background And Preliminaries	2
4	Cryptographic Primitives Overview	2
4.1	Register-based	3
4.2	ECIES + AES-GCM	4
4.3	Re-Encryption	5
4.4	Groth16 SNARK Verification	6
5	Protocol Overview	7
5.1	Design Goals And Requirements	7
5.2	On-Chain And Off-Chain Architecture	8
5.3	Key Management And Identity	9
5.4	Protocol Specification	9
6	Security Model	12
6.1	Assumptions	12
6.2	Trust Model	13
6.3	Threat Analysis	13
6.4	Metadata Leakage	13
6.5	Limitations And Risks	14
6.6	Performance And On-Chain Cost	15
7	Conclusion	15
A	Appendix A - Data Structures	16
B	Appendix B - Proofs	23
	Bibliography	25

1 Abstract

In this report, we introduce the PEACE protocol, an ECIES-based, multi-hop, unidirectional proxy re-encryption scheme for the Cardano blockchain. PEACE solves the encrypted-NFT problem by providing a decentralized, open-source protocol for transferable encryption rights, enabling creators, collectors, and developers to manage encrypted NFTs without relying on centralized decryption services. This work fills a significant gap in secure, private access to NFTs on Cardano. Project Catalyst¹ funded the PEACE protocol in round 14.

2 Introduction

The encrypted NFT problem refers to the lack of inherent encryption and security measures for the digital assets, making the associated content vulnerable to unauthorized access, duplication, and loss, and is one of the most significant issues with current NFT standards on Cardano. Either the data is not encrypted, available to everyone who views the NFT, or the encryption requires centralization, with a company handling it on behalf of users. Current solutions [1] claim to offer decentralized encrypted assets (DEAs), but lack a publicly available, verifiable cryptographic protocol or an open-source implementation. Most, if not all, of the mechanics behind current DEA solutions remain undisclosed. This report aims to fill that knowledge gap by providing an open-source implementation of a decentralized re-encryption protocol for encrypted assets on Cardano.

Several mandatory requirements must be satisfied for the protocol to function as intended. The encryption protocol must allow tradability of both the NFT itself and the right to decrypt the NFT data, implying that the solution must involve smart contracts and a form of encryption that allows data access to be re-encrypted for another user without revealing the encrypted content in the process. The contract side of the protocol should be reasonably straightforward. It needs a way to trade a token that holds the encrypted data and allows other users to receive it. To ensure decryptability, the tokens must be soulbound. On the encryption side of the protocol is some form of encryption that enables the re-encryption process to function correctly. Luckily, this type of encryption has been in cryptography research for quite some time [2] [3] [4]. There are even patented cloud-based solutions already in existence [5]. Currently, there are no open-source, fully on-chain, decentralized re-encryption protocols for encrypting NFT data on Cardano. The PEACE protocol provides an open-source solution to this problem.

The PEACE protocol implements a unidirectional, multi-hop proxy re-encryption (PRE) scheme [6] that is ECIES-based [7] and uses AES [8]. Unidirectionality means that Alice can re-encrypt for Bob, and Bob can then re-encrypt it back to Alice, using different encryption keys. Unidirectionality is important for tradability, as it defines the one-way flow of data and removes any restriction on who can purchase the NFT. Multi-hop means that the flow of encrypted data from Alice to Bob to Carol, and so on, does not end, in the sense that it cannot be re-encrypted for a new user. Multi-hopping is important for tradability, as a finitely tradable asset does not fit many use cases. Typically, an asset should always be tradable if the user wants to trade it. The encryption primitives used in the protocol are considered industry standards at the time of this report.

The remainder of this report is as follows. Section 2 discusses the preliminaries and background required for this project. Section 3 provides a brief overview of the required cryptographic primitives. Section 4 gives a detailed description of the protocol. Section 5 examines security and threat analysis,

¹<https://projectcatalyst.io/funds/14/cardano-use-cases-concepts/decentralized-on-chain-data-encryption>

the protocol's limitations, and related topics. The goal of this report is to serve as a comprehensive reference and description of the PEACE protocol.

3 Background And Preliminaries

Understanding the protocol will require some technical knowledge of modern cryptographic methods, a basic understanding of elliptic curve arithmetic, and a general understanding of how smart contracts work on the Cardano blockchain. Anyone comfortable with these topics will find this report very useful and easy to follow. The report will attempt to use research standards for terminology and notation. The elliptic curve used in this protocol will be BLS12-381 [9]. Aiken is used to write all smart contracts required for the protocol [10].

Table 1: Symbol Description [11]

Symbol	Description
n	The order of $E(\mathbb{F}_p)$
r	A prime number dividing n
δ	A non-zero integer in $\mathbb{Z}_n$
$\mathbb{G}_1$	A subgroup of order r of $E(\mathbb{F}_p)$
$\mathbb{G}_2$	A subgroup of order r of the twist $E'(\mathbb{F}_{p^2})$
$\mathbb{G}_T$	The multiplicative target group of the pairing: $\mu_r \subset \mathbb{F}_{p^{12}}$
$e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$	A type-3 bilinear pairing
q	A fixed generator in $\mathbb{G}_1$
p	A fixed generator in $\mathbb{G}_2$
R	The Fiat-Shamir transformer
H	A hash function
m	The order of Ed25519
γ	A non-zero integer in $\mathbb{Z}_m$

The protocol, including both on-chain and off-chain components, will heavily utilize the **Register** type shown in Listing 1. The **Register** stores a generator, $g \in \mathbb{G}_\kappa$ and the corresponding public value $u = [\delta]g$ where $\delta \in \mathbb{Z}_n$ is a secret. We shall assume that the hardness of ECDLP and CDH in $\mathbb{G}_1$ will result in the inability to recover the secret δ . When using a pairing, we additionally rely on the standard bilinear Diffie-Hellman assumptions over the subgroups $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$. We will represent the groups $\mathbb{G}_1$ and $\mathbb{G}_2$ with additive notation and $\mathbb{G}_T$ with multiplicative notation.

Where required, we will verify Ed25519 signatures [12] for cost-minimization, as relying solely on pure BLS12-381 for simple signatures becomes too costly on-chain. There will be instances where the Fiat-Shamir transform [13] will be applied to a Σ -protocol to transform it into a non-interactive variant. In these cases, the hash function will be Blake2b-224 [14]. For the SNARK circuit internals, the protocol uses MiMC as the hash function for hashing $\mathbb{G}_T$ elements to scalars in $\mathbb{Z}_n$, as MiMC is efficient in arithmetic circuits.

4 Cryptographic Primitives Overview

This section provides brief explanations of the cryptographic primitives required by the protocol. If a primitive has an algorithmic description, then it will be included in the respective sub-section.

We will represent the **Register** type as a tuple, (g, u) , for simplicity inside the algorithms. We shall assume that the compression and uncompression of the elliptic curve points are canonical and follow the Zcash implementation [15]. Correctness proofs for many algorithms are in Appendix B.

4.1 Register-based

The protocol requires proving knowledge of a user's secret using a Schnorr Σ -protocol [16] [17]. This algorithm is both complete and zero-knowledge. We can use simple Ed25519 signatures for spendability, and then utilize the Schnorr Σ -protocol for knowledge proofs related to encryption. We will make the protocol non-interacting via the Fiat-Shamir transform, R , as shown in Listing 18.

Algorithm 1: Non-interactive Schnorr's Σ -protocol for the discrete logarithm relation

Input:

(g, u) where $g \in \mathbb{G}_\kappa$, $u = [\delta]g \in \mathbb{G}_\kappa$

Output: bool

- 1 select a random $r \in \mathbb{Z}_n$
 - 2 compute $a = [r]g$
 - 3 calculate $c = R(g, u, a)$
 - 4 compute $z = \delta \cdot c + r$
 - 5 output $[z]g = a + [c]u$
-

The protocol requires proving a binding relationship between a user's public value and other known elliptic-curve elements. The binding proof is a combination of multiple Schnorr's Σ -protocols. The value χ is the $\mathbb{G}_1$ element of the second term in the encryption level, specifically, `r2_g1b`.

Algorithm 2: Non-interactive Binding Σ -protocol

Input:

(g, u) where $g \in \mathbb{G}_1$, $u = [\delta]g \in \mathbb{G}_1$

(r_1, χ) where $r_1 \in \mathbb{G}_1$, $\chi \in \mathbb{G}_1$

(a, r) where $a \in \mathbb{Z}_n$ and $r \in \mathbb{Z}_n$

Output: bool

- 1 select a random $\rho \in \mathbb{Z}_n$ and $\alpha \in \mathbb{Z}_n$
 - 2 compute $t_1 = [\rho]g$
 - 3 compute $t_2 = [\alpha]g + [\rho]u$
 - 4 calculate $c = R(g, u, t_1, t_2)$
 - 5 compute $z_a = a \cdot c + \alpha$
 - 6 compute $z_r = r \cdot c + \rho$
 - 7 output $[z_a]g + [z_r]u = t_2 + [c]\chi \wedge [z_r]g = t_1 + [c]r_1$
-

There will be times when the protocol requires proving some equality using pairings. In these cases, we can use something akin to the BLS signature scheme, allowing only someone with the knowledge of the secret to prove the pairing equivalence. BLS signatures are a straightforward yet important signature scheme for the protocol, as they enable public confirmation of knowledge of a complex relationship beyond the limitations of Schnorr's Σ -protocols. BLS signatures work because of the bilinearity of the pairing [18].

Algorithm 3: Boneh-Lynn-Shacham (BLS) signature method

Input:

(g, u, c, w, m) where $g \in \mathbb{G}_1$, $u = [\delta]g \in \mathbb{G}_1$,
 $c = p^{H(m)} \in \mathbb{G}_2$, $w = [\delta]c \in \mathbb{G}_2$, and $m \in \{0, 1\}^*$

Output: bool

- 1 $e(u, c) = e(g, w)$
 - 2 $e(q^\delta, c) = e(q, c^\delta)$
-

4.2 ECIES + AES-GCM

The Elliptic Curve Integrated Encryption Scheme (ECIES) is a hybrid protocol involving asymmetric cryptography with symmetric ciphers. The encryption used in ECIES is the Advanced Encryption Standard (AES). ECIES and AES, combined with a key derivation function (KDF) such as HKDF [19], form a complete encryption system. For readability, we present a simplified ECIES sketch here. The reference implementation provides the exact PEACE instantiation and serialization rules.

Algorithm 4: Encryption using ECIES + AES

Input:

(g, u) where $g \in \mathbb{G}_\kappa$, $u = [\delta]g \in \mathbb{G}_\kappa$, $m \in \{0, 1\}^*$

Output: (r, c, h)

- 1 select a random $\rho \in \mathbb{Z}_n$
 - 2 compute $r = [\rho]g$
 - 3 compute $s = [\rho]u$
 - 4 generate $k = KDF(s|r)$
 - 5 encrypt $c = AES(m, k)$
 - 6 compute $h = BLAKE2B(m)$
 - 7 output (r, c, h)
-

Decrypting the ciphertext requires rebuilding the data encryption key (DEK), k , from the KDF. The DEK is rebuildable because r is public and the user knows the secret δ , allowing them to decrypt the data.

Algorithm 5: Decryption using ECIES + AES

Input:

(g, u) where $g \in \mathbb{G}_1$, $u = [\delta]g \in \mathbb{G}_1$,
(r, c, h) as the capsule

Output: ($\{0, 1\}^*$, bool)

- 1 compute $s' = [\delta]r$
 - 2 generate $k' = KDF(s'|r)$
 - 3 compute $m' = AES(c, k')$
 - 4 compute $h' = BLAKE2B(m')$
 - 5 output ($m', h' = h$)
-

Algorithm 4 describes the case where a **Register** is used to generate the DEK from the KDF function. Anyone with knowledge of k may decrypt the ciphertext. The algorithm shown differs slightly from the PEACE protocol's implementation, as the protocol allows transferring the DEK to another **Register**; however, the general flow remains the same. The key takeaway here is that encrypting a message and decrypting the resulting ciphertext require a key to generate the DEK.

Both algorithms 4 and 5 use a simple hash function for authentication. In the PEACE protocol, we will use AES-GCM with authenticated encryption with associated data (AEAD) for authentication.

4.3 Re-Encryption

There are various types of re-encryption schemes, ranging from classical proxy re-encryption (PRE) to hybrid methods. These re-encryption schemes involve a proxy, an entity that performs both re-encryption and verification. The PRE used in the PEACE protocol is modeled as an interactive flow between the current owner and a prospective buyer, utilizing a smart contract as part of the proxy. We need an interactive scheme because, in many real-world use cases, numerous off-chain checks, such as KYC/AML regulations and various legal requirements, must occur before transferring the right to decrypt on-chain to the new owner. The PEACE protocol obtains interactivity via a bidding system that requires the current owner to agree to the exchange.

The method described below is a hybrid approach. The current owner's wallet performs the re-encryption process for the buyer. At the same time, the Cardano smart contract acts as a proxy, verifying various cryptographic proofs, enforcing the correct bindings, handling payments, and updating the on-chain owner fields. This design explicitly supports off-chain processes before the transfer of decryption rights. The current owner only submits the re-encryption transaction once these off-chain conditions are satisfied. This method will support the broadest range of real-world asset use cases. The PRE is unidirectional, meaning the re-encryption flow is one-way: from the current owner to the next owner. If Alice delegates to Bob, Bob does not automatically gain the ability to 'go backwards' and create ciphertexts for Alice using the same re-encryption material. This flow differs from a bidirectional method, where the PRE is symmetric, enabling a two-way encryption relationship between the parties, meaning Alice can transform a ciphertext into one for Bob, and Bob can transform a ciphertext into one for Alice, without either Alice or Bob having to re-run the entire re-encryption flow. That is not what we want for this implementation. Each direction is a separate, explicit transfer of rights with its own re-encryption material, meeting the protocol's tradability requirements.

Note that in the original Catalyst proposal, the protocol defines itself as a bidirectional, multi-hop PRE. However, during the design phase, it became clear that the actual Cardano use case requires a unidirectional, multi-hop PRE. This change is fully compatible with the original proposal's PRE goals (transfer of decryption rights without exposing plaintext or private keys), but reflects the reality of trading tokens via Cardano smart contracts within the PRE landscape.

Algorithm 6: Owner-mediated re-encryption from Alice to Bob

Input: (q, u) where $q \in \mathbb{G}_1$, $u = [\delta_a]q \in \mathbb{G}_1$ (Alice's public key), (q, v) where $v = [\delta_b]q \in \mathbb{G}_1$ (Bob's public key),Alice's secret key $\delta_a \in \mathbb{Z}_n$ $p \in \mathbb{G}_2$ Alice's current **HalfEncryptionLevel**: $(r_{1,a}, r_{2,a}, r_{4,a})$, where $r_{1,a}, r_{2,a} \in \mathbb{G}_1$ and $r_{4,a} \in \mathbb{G}_2$
 (h_0, h_1, h_2) , where $h_i \in \mathbb{G}_2$ are fixed public points.**Output:** Bob's **HalfEncryptionLevel**: $(r_{1,b}, r_{2,b}, r_{4,b})$ Alice's **FullEncryptionLevel**: $(r_{1,a}, r_{2,a}, r_5, r_{4,a})$

- 1 select a random $a \in \mathbb{Z}_n$
 - 2 compute $\kappa = e(q^a, h_0)$
 - 3 select a random $r \in \mathbb{Z}_n$
 - 4 compute $r_{1,b} = [r]q$
 - 5 compute $r_{2,b} = [a]q + [r]v \in \mathbb{G}_1$
 - 6 compute $c = [\text{BLAKE2b}(r_{1,b})]h_1 + [\text{BLAKE2b}(r_{1,b}||r_{2,b})]h_2$
 - 7 compute $r_{4,b} = [r]c$
 - 8 compute $r_5 = [H(\kappa)]p + [\delta_a](-h_0) \in \mathbb{G}_2$
 - 9 output $(r_{1,b}, r_{2,b}, r_{4,b})$ and $(r_{1,a}, r_{2,a}, r_5, r_{4,a})$
-

Algorithm 6 describes the actual re-encryption process for Alice, resulting in the transfer of the decryption rights to Bob. Bob can then use this information to recursively calculate the secret κ and eventually the original secret used in the encryption process.

4.4 Groth16 SNARK Verification

The protocol uses a Groth16 zero-knowledge proof to ensure that the re-encryption witness $W = q^{H(\kappa)}$ is correctly derived from the secret $\kappa = e(q^a, h_0)$. The SNARK circuit proves the following statement without revealing the secrets a or r :

Algorithm 7: SNARK Circuit Statement

Input:Secret inputs: (a, r) where $a, r \in \mathbb{Z}_n$ Public inputs: (v, w_0, w_1) where $v, w_0, w_1 \in \mathbb{G}_1$ **Output:** bool

- 1 compute $\kappa = e([a]q, h_0)$
 - 2 compute $hk = \text{MiMC}(\kappa||\text{DomainTag})$
 - 3 compute $p_0 = [hk]q$
 - 4 compute $p_1 = [a]q + [r]v$
 - 5 output $w_0 = p_0 \wedge w_1 = p_1$
-

The public inputs v , w_0 , and w_1 are provided as affine $\mathbb{G}_1$ coordinates. The on-chain verifier receives these coordinates as 64-bit limbs and reconstructs the compressed $\mathbb{G}_1$ points via limb compression verification. This binding ensures that the SNARK public inputs correspond to the actual elliptic curve points stored in the datum.

The Groth16 verifier uses the gnark commitment extension, which adds a Pedersen commitment proof-of-knowledge check alongside the standard pairing verification. The verification equation is:

$$e(A, B) \cdot e(vk_x, -\gamma) \cdot e(C, -\delta) \cdot e(\alpha, -\beta) = 1$$

where vk_x incorporates both the public inputs and commitment wires.

5 Protocol Overview

The PEACE protocol is an ECIES-based, multi-hop, unidirectional proxy re-encryption scheme for the Cardano blockchain, allowing creators, collectors, and developers to trade encrypted NFTs without relying on centralized decryption services. The current implementation uses the Cardano blockchain as the data storage layer. Cardano’s chain parameters limit on-chain storage to less than 16 KB. For use cases requiring larger payloads, the data storage layer can be extended to support arbitrary file sizes.

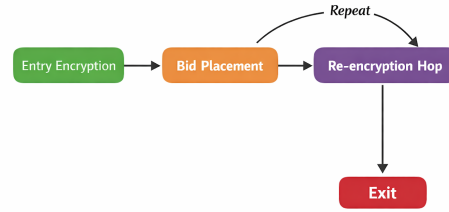


Figure 1: Protocol state flow

5.1 Design Goals And Requirements

Two equally important areas, the on-chain and off-chain, define the protocol design. The on-chain design is everything related to smart contracts written in Aiken for the Cardano blockchain. The off-chain design includes transaction construction, cryptographic proof generation, and the happy-path flow. The design on both sides will focus on a two-party example: Alice and Bob, who want to trade encrypted data. Alice will be the original owner, and Bob will be the new owner. The current off-chain implementation focuses on the two-party case. Extending to a general n-party system is planned as future work.

The protocol must allow continuous trading via a multi-hop PRE, meaning that Alice trades with Bob, who can then trade with Carol. In this setting, Alice will trade to Bob, then Bob will trade back to Alice, rather than Carol, without any loss of generality. Each hop will generate a new owner and decryption data for the encryption UTxO. The on-chain storage remains constant per hop: the datum stores only the current half-level and the previous full-level, rather than a growing list of all levels. Users reconstruct the complete encryption level history by querying the blockchain from the mint transaction to the current state. Users will use a basic bid system for token trading. A user may choose not to trade their token by simply not selecting a bid if one exists.

The protocol implements a state machine with two states: **Open** and **Pending**. In the **Open** state, the owner may remove the encryption UTxO, submit a SNARK proof to transition to **Pending**, or simply hold the asset. In the **Pending** state, a bidder’s re-encryption request is awaiting completion; the owner may complete the re-encryption (transitioning back to **Open** with a new owner), cancel

the pending request, or allow the request to expire after a time-to-live (TTL) threshold. This state machine ensures atomicity of the SNARK verification and re-encryption steps while allowing recovery from abandoned transactions.

The re-encryption process needs to flow in one direction per hop. Alice trades with Bob, and that is the end of their transaction. Bob does not gain the ability to re-encrypt the data back to Alice without a new bid from Alice, which restarts the re-encryption process. Any bidirectionality here implies symmetry between Alice and Bob, thereby circumventing the re-encryption requirement via token trading. The unidirectional requirement forces tradability to follow the typical trading interactions currently found on the Cardano blockchain.

Each UTxO in this system must be uniquely identified via an NFT. The uniqueness requirement works well for the encryption side because the NFT could be a tokenized representation of the encrypted data, something similar to a CIP68 [20] contract, but using a single token. The bid side does work, but the token becomes a pointer rather than having any real data attached, essentially a unique, one-time-use token. Together, they provide the correct uniqueness requirement. UTxOs may be removed from the contract at any time by the owner. After the trade, the owner of the encrypted data may do whatever they want with that data. The protocol does not require the re-encryption contract to permanently store the encrypted data.

The protocol will use an owner-mediated re-encryption flow (a hybrid PRE), which is UX-equivalent to a classical proxy re-encryption scheme in this setting, since smart contracts on Cardano are passive validators and do not initiate actions. Ultimately, some user must act as the proxy, the one doing the re-encryption, because the contract cannot do it on its own. The smart contract must act as the proxy's validator, not solely as the proxy itself. In the current implementation, the owner acts as their own proxy in the protocol.

5.2 On-Chain And Off-Chain Architecture

There will be two user-focused smart contracts: one for re-encryption and the other for bid management. Any UTxO inside the re-encryption contract is for sale via the bidding system. A user may place a bid into the bid contract, and the current owner of the encrypted data may select it as payment for re-encrypting the data to the new owner. To ensure functionality, a reference data contract must exist, as it resolves circular dependencies. The reference datum will contain the script hashes for the re-encryption and bid contracts. Data structures and functions are in Appendix A.

The bid contract datum structure is shown in Listing 2. The bid datum contains all of the required information for re-encryption. The owner of a bid UTxO will be type **Register** in $\mathbb{G}_1$. The **pointer** is the NFT name on the bid UTxO, and **token** is the NFT name on the re-encryption UTxO. The **token** forces the bid to only apply to a specific sale.

The bid contract redeemer structures are shown in Listing 3. Entering into the bid contract uses the **EntryBidMint** redeemer, triggering a **pointer** mint validation, a **token** UTxO existence check, an Ed25519 signature with **owner_vkh**, and a Schnorr Σ -protocol using **owner_g1**. Leaving the bid contract requires using **RemoveBid** and **LeaveBidBurn** redeemers together, triggering a **pointer** burn validation and Ed25519 signature with **owner_vkh**. When a user selects a bid, they will use **UseBid** and **LeaveBidBurn** together, triggering a **pointer** burn validation and the proxy re-encryption validation.

Listing 4 shows the re-encryption contract datum structure. The re-encryption datum contains all of the required information for decryption. The owner of the re-encryption UTxO will be type

Register in $\mathbb{G}_1$. The **token** is the NFT name on the re-encryption UTxO. The ciphertext and related data are in the **Capsule** subtype. Rather than storing all encryption levels, the datum stores only the current **half_level** and optionally the previous **full_level**. Users reconstruct the complete level history by querying the blockchain. The **status** field tracks the state machine position: **Open** for normal operation, or **Pending** with the SNARK public inputs and TTL when awaiting re-encryption completion.

The re-encryption contract redeemer structures are shown in Listing 5. Entering into the re-encryption contract uses the **EntryEncryptionMint** redeemer, triggering a **token** mint validation, an Ed25519 signature with **owner_vkh**, a binding proof using **owner_g1**, and a Schnorr Σ -protocol using **owner_g1**. Leaving the re-encryption contract requires using both **RemoveEncryption** and **LeaveEncryptionBurn** redeemers, triggering a **token** burn validation and an Ed25519 signature with **owner_vkh**. The **UseSnark** redeemer initiates the re-encryption process by verifying the Groth16 proof and transitioning the status from **Open** to **Pending**. The **UseEncryption** redeemer completes the re-encryption by verifying limb compression, updating ownership, and transitioning back to **Open**. The **CancelEncryption** redeemer allows the owner to cancel a pending re-encryption or allows automatic cancellation after TTL expiration.

The re-encryption flow requires two transactions: first **UseSnark** to submit the SNARK proof, then **UseEncryption**, **UseBid**, and **LeaveBidBurn** together to complete the transfer.

5.3 Key Management And Identity

Each user in the protocol can deterministically generate BLS12-381 key pairs represented by **Register** value in $\mathbb{G}_1$. The $\mathbb{G}_1$ points are the user's on-chain identity for encryption and signature verification. The corresponding secret scalar, $\delta \in \mathbb{Z}_n$, is held off-chain by the user's wallet or client software and never published on-chain.

The BLS12-381 keys used for re-encryption are logically separate from the Ed25519 keys used to sign Cardano transactions. A wallet must manage both Ed25519 keys for authorizing UTxO spending and BLS12-381 scalars for obtaining and delegating decryption rights. Losing or compromising the BLS12-381 secret key means losing the ability to decrypt any items associated with that identity, even if the Cardano spending keys are still available.

The current version does not implement a full key rotation or revocation mechanism. If a user's BLS12-381 secret key is compromised, an attacker can decrypt all current and future capsules addressed to that key, but cannot retroactively remove or alter on-chain history. Handling key rotation, partial recovery, and revocation across many encrypted positions is left as future work.

For each encrypted item, the protocol generates a fresh KEM used at each encryption level. The KEM is never directly stored on-chain. The on-chain Capsule contains the AES-GCM nonce, associated data, and ciphertext.

5.4 Protocol Specification

5.4.1 Phase 1: Creating The Encryption UTxO

The protocol flow starts with Alice selecting a secret $[\gamma] \in \mathbb{Z}_m$ and $[\delta] \in \mathbb{Z}_n$. The secret γ will generate an Ed25519 keypair. The secret δ will generate the **Register** in $\mathbb{G}_1$ using the fixed generator, q . Alice will fund the address associated with the **VerificationKeyHash**, the **vkh**, of the Ed25519 keypair with enough Lovelace to pay for the minimum required Lovelace for the contract

UTxO, the change UTxO, and the transaction fee. Alice may then build the re-encryption entry transaction.

The re-encryption entry transaction will contain a single input and two outputs. The transaction will mint a `token` using the `EntryEncryptionMint` redeemer. The `token` name is generated by the concatenation of the input's output index and transaction ID, as shown in the Listing 6. The protocol specification assumes a single input, but this transaction may use multiple inputs. If more than one input exists, then the first element of a lexicographically sorted input list will be used for the name generation.

Alice may now finish building the `EncryptionDatum` by constructing the `half_level` and `capsule` fields. Since Alice is the first owner, she will encrypt it for herself. Alice will encrypt the original data by generating a root secret $\kappa_0 \in \mathbb{G}_T$. The root secret, κ_0 , will be used in the KDF to produce a valid DEK. The message will be encrypted using AES-GCM. The `Capsule` type will store the resulting information. Listing 7 is a Pythonic pseudocode for generating the original encrypted data and the first encryption level. The sub-types of the `EncryptionDatum` can be populated as shown in Listing 8. The contract will validate the first half-level using the assertion from Listing 9. Alice can prove to herself that the encryption level is valid by verifying the assertion in Listing 10. Alice may now construct the full `EncryptionDatum` as shown in Listing 11, with `status` set to `Open` and `full_level` set to `None`.

The entry redeemer verifies that Alice's `owner_vkh` is valid via an Ed25519 signature, since Alice needs a valid `vkh` to remove the entry. The entry redeemer also verifies a valid `Register` via a Schnorr Σ -protocol as shown in Algorithm 1. Alice needs this to decrypt her own data and to verify that she binds her public value to the first encryption level via a binding proof, as shown in Algorithm 2. The encrypted data is ready to be traded after successfully creating a valid entry transaction and submitting it to the Cardano blockchain.

While the encryption UTxO's `status` remains `Open`, Alice may update her asking price using the `UpdateEncryptionPrice` spend redeemer. This requires Alice's signature and enforces that the new price is non-negative. Only the `new_price` field may change; all other datum fields and non-ADA tokens on the UTxO must remain constant. This allows Alice to adjust her asking price without recreating the encryption UTxO.

5.4.2 Phase 2: Creating The Bid UTxO

Bob may now create a bid UTxO in the bid contract to purchase the encrypted data from Alice. First, Bob selects a secret $[\gamma] \in \mathbb{Z}_m$ and $[\delta] \in \mathbb{Z}_n$. Similarly to Alice, the secret γ will generate an Ed25519 keypair and the secret δ will generate the `Register` in $\mathbb{G}_1$ using the fixed generator, q . Bob will fund the address associated with his `vkh` with enough Lovelace to pay for payment, the change UTxO, and the transaction fee. Bob may then build the bid entry transaction. Note that the on-chain datum size remains constant regardless of hop count, as only the current half-level and previous full-level are stored. Bob should contribute to the minimum required Lovelace for the encrypted data, though this is not an on-chain requirement.

The structure of the bid entry transaction is similar to that of the re-encryption entry transaction, but uses `EntryBidMint` instead of `EntryEncryptionMint`. The transaction input derives the `pointer` token name in the same way as the `token` name. A user may reference the `token` name on-chain from the re-encryption contract. Bob may then create the `BidDatum` as shown in Listing 12.

Similar to the re-encryption contract, the entry redeemer will verify Bob's `vkh` and the `Register`

values in $\mathbb{G}_1$. A valid **Register** is important, as the validity of the $\mathbb{G}_1$ point determines whether Bob can decrypt the data after the re-encryption process. The **new_price** field in the **BidDatum** represents the price Bob is willing to pay for Alice to re-encrypt the data to his **Register**. There may be many bids, but Alice may only select a single bid for the re-encryption transaction. Bob may update the price of an existing bid using the **UpdateBidPrice** spend redeemer, which requires Bob's signature and enforces that the new price is non-negative. Only the **new_price** field may change; all other datum fields and non-ADA tokens on the UTxO must remain constant.

To mitigate grief attacks during the re-encryption flow, each bid is automatically locked for a minimum period (**minimum_bid_lock**, currently 6 hours) upon creation. The on-chain **EntryBidMint** redeemer enforces that **locked_until** is at least **minimum_bid_lock** in the future. The **RemoveBid** spend redeemer enforces that the transaction's validity interval lower bound exceeds **locked_until**. This guarantees Alice a safe window to compute the SNARK proof and submit both re-encryption transactions without Bob withdrawing his bid mid-process.

5.4.3 Phase 3: SNARK Submission And Re-Encryption

The re-encryption process occurs in two transactions to accommodate the computational cost of SNARK verification.

Transaction 1: SNARK Submission. Alice generates a Groth16 proof demonstrating that the witness $W = q^{H(\kappa)}$ is correctly derived from the pairing secret $\kappa = e(q^a, h_0)$. Alice submits this proof using the **UseSnark** redeemer, which verifies the Groth16 proof (via the **groth_witness** withdraw validator) and the commitment proof-of-knowledge. Upon successful verification, the datum's **status** transitions from **Open** to **Pending(groth_public, ttl)**, where **groth_public** contains the SNARK public inputs (the limb representation of the $\mathbb{G}_1$ points) and **ttl** is the expiration time.

Transaction 2: Re-Encryption Completion. Alice selects a bid UTxO from the bid contract and completes the re-encryption using the **UseEncryption** redeemer together with **UseBid** and **LeaveBidBurn**. This step burns Bob's bid token, updates the on-chain data to Bob's ownership, and creates the re-encryption proofs. The contract verifies limb compression to ensure the SNARK public inputs match the actual $\mathbb{G}_1$ points (**bid_owner_g1.public_value**, the witness, and **next_half_level.r2_g1b**). The PRE proofs demonstrate that the values produced by the re-encryption process match the expected values via pairings involving the original owner's **Register**, the new owner's **Register**, and the next encryption level. Additionally, the bid's **new_price** is transferred to the encryption datum's **new_price** field, recording the agreed-upon price on the encryption UTxO. If everything is consistent, then ownership and decryption rights transfer to Bob, and the status returns to **Open**.

Listing 13 is a Pythonic pseudocode for generating the next encryption level. Bob's half-level and Alice's full-level are shown in Listing 14. The complete next encryption datum is shown in Listing 15.

The contract will validate the re-encryption using a binding proof and two pairing proofs as shown in Listing 16. The first pairing assertion follows Alice's first-level validation, ensuring that the encryption level terms are consistent. The second pairing assertion shows that Alice correctly created the r_5 term. The SNARK proves that the secret κ truly does equal the witness when hashed, $W = q^{H(\kappa)}$, where $H(\kappa)$ and κ are secrets and W is public.

5.4.4 Phase 4: Decryption

Bob can now decrypt the root key by recursively computing all the random $\mathbb{G}_T$ points as shown in Listing 17. Since the on-chain datum stores only the current half-level and previous full-level, Bob must first reconstruct the complete encryption level history by querying the blockchain for all transactions from the token’s mint to the current state. Each historical transaction contains the half-level and full-level at that point in time, allowing Bob to rebuild the complete list of encryption levels needed for recursive decryption.

6 Security Model

The PEACE protocol needs to have reasonable security. In a real-world production setting, the protocol must have a minimal attack surface.

6.1 Assumptions

This protocol inherits standard assumptions from public-key cryptography and public blockchains. The assumptions below describe what must hold for the security claims in this document to be meaningful.

- All compressed curve points accepted by the protocol (on-chain or off-chain) MUST be validated as canonical encodings. As members of the correct prime-order subgroup (rejecting non-canonical encodings, the point at infinity where disallowed, and any point not in the intended subgroup), otherwise an attacker may exploit small-subgroup/cofactor edge cases to bypass security claims or forge relations that appear to verify.
- Cryptographic assumptions hold: The security of the construction relies on standard hardness assumptions for the chosen primitives (pairing groups / discrete log), collision resistance and preimage resistance of the hash functions used (including domain separation), and unforgeability of any signature schemes used.
- Correct domain separation: All hashes used for hashing-to-scalar, Fiat–Shamir transcripts, and key derivations use fixed domain tags and unambiguous encodings. A domain-separation bug is a critical security failure.
- Well-formed randomness: All secret scalars and nonces are sampled with high entropy and never reused where uniqueness is required. Randomness failures (poor RNG, nonce reuse, low-entropy secrets) are catastrophic.
- Endpoint key safety: Alice’s and Bob’s long-term secret keys must remain confidential. Extracting keys from the wallet/device undermines the confidentiality and authenticity guarantees.
- On-chain validation is authoritative: The ledger enforces the validator exactly as written (Aiken/Plutus semantics). Any check performed only off-chain is advisory and not part of the security boundary.
- Proof system assumptions: In a production setting, the protocol uses SNARKs. Their required assumptions hold (soundness and any additional properties needed for adversarial settings). If the SNARK requires a trusted setup, then the corresponding trapdoor (“toxic waste”) is assumed destroyed.
- Chain security: The blockchain provides finality and censorship-resistance to the degree

commonly assumed for Cardano. Prolonged reorgs, validator bugs, or sustained censorship are out of scope.

- Scope boundary: The protocol does not assume (and does not attempt to enforce) that the plaintext has any particular meaning or quality, nor does it assume Bob will keep plaintext private after decryption.

6.2 Trust Model

The protocol design minimizes trust between Alice and Bob. The smart contract is the source of truth for what constitutes a valid re-encryption hop. Anything not enforced by the on-chain validator is advisory. We do not assume Alice or Bob is honest. Either party may attempt to cheat, submit malformed data, or abort mid-protocol. The proxy is at best semi-trusted: it may be offline, malicious, or compromised. Any compromised keys are a severe failure. We assume standard cryptographic hardness of the underlying primitives (pairings / discrete log, hash collision resistance, and signature unforgeability), and we assume the wallet or OS will protect endpoint key material. A compromise of long-term keys (Alice/Bob/etc) is out of scope except where explicitly mitigated (e.g., domain separation and on-chain binding checks).

6.3 Threat Analysis

Adversary capabilities

- Full network observer: can read all on-chain data, replay transactions, and correlate timing/amount patterns.
- Active attacker: can submit arbitrary transactions, craft malformed ciphertext/proofs, and attempt to use validator failures/success as an oracle.
- Insider attacker: Alice or Bob may act maliciously (sell garbage, withhold finalization, attempt to claim funds without delivering the correct re-encryption).
- Key compromise: theft of a participant's secret keys is possible.

Primary threats and production mitigations

- Invalid re-encryption accepted on-chain: mitigated by strict on-chain validation that binds ciphertext components, public keys, and transcript hashes to the expected relations.
- Related-ciphertext / CCA-style probing: mitigated by making ciphertexts and re-encryption steps non-malleable under the on-chain checks (proofs must bind all relevant fields so "tweaks" are rejected).
- Multi-hop bypass: if downstream decryption reveals intermediate artifacts used to decrypt upstream ciphertexts without the proxy, it breaks the intended delegation boundary; mitigation is hop-level re-randomization / re-encapsulation and careful design to ensure decryption yields only plaintext (not reusable upstream capabilities).
- Fairness failure (abort/grief): either party can stop cooperating; mitigations are economic and protocol flow design, not cryptography alone.

6.4 Metadata Leakage

The protocol runs on a public UTxO ledger, so metadata leakage is unavoidable.

Potential leakage includes:

- Transaction graph linkage: repeated verification keys, registers, UTxO patterns, and timing can link multiple protocol runs to the same actors or workflow.
- Protocol fingerprinting: ciphertext sizes, hop count, and datum/redeemer structure can reveal which step the protocol is in and correlate participants across transactions.
- Value and timing leakage: amounts, fees, and time between steps can reveal trade size, urgency, and repeated counterparties.
- Key/identity linkage via commitments: even with hashed values, fixed-format commitments and domain tags can still be fingerprinted if reused or if inputs have low entropy.

Mitigations are partial and operational:

- Rotate addresses and avoid stable identifiers where possible
- Treat privacy as a separate layer (mixing, batching, relayers) rather than something the core protocol guarantees.

6.5 Limitations And Risks

- The protocol includes a Groth16 SNARK that proves the published $H(\kappa)$ -dependent terms are derived from the actual pairing secret $\kappa = e(q^{a_0}, h_0)$. This SNARK enforces $hk = \text{MiMC}(e(q^{a_0}, h_0) \parallel \text{DomainTag})$ and $W = q^{hk}$ without revealing κ , hk , or a_0 . The SNARK uses MiMC for hashing within the circuit, as it is efficient in arithmetic circuits. The limb compression verification binds the SNARK public inputs to the actual on-chain G1 points.
- The protocol can prove key-binding and correct re-encryption relations, but it cannot prove that the encrypted content is “valuable” or matches an off-chain description. Disputes about semantics require external mechanisms.
- The protocol does not protect the data after decryption. If Bob decrypts the plaintext, Bob can copy or leak it. Cryptography cannot prevent exfiltration. Only economic or legal controls can reduce this risk.
- The protocol does not ensure a perfectly fair exchange. Either party can abort at different stages. The two-transaction re-encryption flow introduces a window where Alice has submitted the SNARK proof but not yet completed the transfer. The TTL-based cancellation mechanism provides recovery but does not guarantee atomicity. To mitigate grief attacks where a buyer removes a bid during Alice’s SNARK computation, bids are automatically locked for a minimum period (`minimum_bid_lock`) upon creation. This lock prevents the `RemoveBid` redeemer from succeeding until the lock expires, giving Alice a guaranteed window to compute and submit her proof. Combined with transaction chaining (submitting both re-encryption transactions back-to-back after offline proof generation), this eliminates the primary grief vector.
- Key compromise is catastrophic. Any theft of secret keys compromises the confidentiality of those assets. Losing secret keys prevents decrypting.
- The protocol aims for CCA security through the combination of binding proofs, pairing checks, and SNARK verification. The SNARK binds the witness computation to the actual pairing secret, preventing arbitrary witness substitution. However, a formal CCA security proof is left as future work.

-
- The protocol does not limit hops directly. However, since the on-chain storage is now constant (only current half-level and previous full-level), the Cardano UTxO size limit no longer constrains the hop count. Users must query the blockchain to reconstruct the full encryption history for decryption.
 - Pairing-heavy verification and SNARK verification can approach the CPU budget, requiring multi-transaction validation flows, increasing complexity, and can reduce UX reliability under network congestion.

6.6 Performance And On-Chain Cost

The re-encryption process performs well within Cardano’s transaction limits by splitting the validation across two transactions. The generation of the SNARK proofs occurs off-chain and takes several seconds on commodity hardware. The encryption setup is straightforward, and the PRE flow is simple to understand.

The first transaction (**UseSnark**) verifies the Groth16 proof and commitment proof-of-knowledge via a withdraw validator. The Groth16 verification involves multiple pairing operations and public input accumulation, consuming a significant portion of the CPU budget. The commitment verification adds additional pairing checks. This separation allows the expensive SNARK verification to occur independently of the re-encryption logic.

The second transaction (**UseEncryption**) verifies limb compression (binding SNARK public inputs to G1 points), binding proofs, and pairing checks for the R5 term. The Schnorr and binding proofs are relatively cheap, but the pairing proofs remain expensive. By separating the SNARK verification into its own transaction, the re-encryption validation stays within budget.

The two-transaction design introduces additional complexity and a brief window where the status is **Pending**. However, this approach ensures the protocol remains feasible on-chain while providing the security guarantees of SNARK-verified witness creation. The constant on-chain storage (only current half-level and previous full-level) also reduces datum size compared to storing all historical levels.

7 Conclusion

The PEACE protocol is a multi-use, unidirectional PRE for the Cardano blockchain with reasonable security guarantees. PEACE emphasizes correctness, composability, and an auditable trust boundary. The integration of a Groth16 SNARK ensures that the re-encryption witness is correctly derived from the pairing secret, addressing the key limitation of earlier designs. The two-transaction flow accommodates on-chain resource limits while maintaining security. We highlight the realities of the UTxO model, metadata leakage, and on-chain resource limits, and show how the design keeps cryptographic enforcement feasible while preserving a clear path toward stronger privacy and robustness. PEACE provides a concrete foundation for encrypted-asset markets on Cardano. It shows what is possible on-chain, what should remain off-chain, and how ownership can evolve across multiple hops while preserving decryption continuity for the rightful holder.

A Appendix A - Data Structures

```
1 pub type Register {
2   // the generator, #<Bls12_381, G1> or #<Bls12_381, G2>
3   generator: ByteArray,
4   // the public value, #<Bls12_381, G1> or #<Bls12_381, G2>
5   public_value: ByteArray,
6 }
```

Listing 1: The Register type

```
1 pub type BidDatum {
2   owner_vkh: VerificationKeyHash,
3   owner_g1: Register,
4   pointer: AssetName,
5   token: AssetName,
6   locked_until: Int,
7   new_price: Int,
8 }
```

Listing 2: The Bid datum type

```
1 pub type BidMintRedeemer {
2   EntryBidMint(SchnorrProof)
3   LeaveBidBurn(AssetName)
4 }
5 pub type BidSpendRedeemer {
6   RemoveBid
7   UseBid
8   UpdateBidPrice(Int)
9 }
10 pub type SchnorrProof {
11   z_b: ByteArray,
12   g_r_b: ByteArray,
13 }
```

Listing 3: The Bid redeemer types

```
1 pub type EncryptionDatum {
2   owner_vkh: VerificationKeyHash,
3   owner_g1: Register,
4   token: AssetName,
5   half_level: HalfEncryptionLevel,
6   full_level: Option<FullEncryptionLevel>,
7   capsule: Capsule,
8   status: Status,
9   new_price: Int,
10 }
11 pub type Capsule {
12   nonce: ByteArray,
13   aad: ByteArray,
14   ct: ByteArray,
15 }
16 pub type Status {
17   Open
18   Pending(GrothPublic, Int)
19 }
20 pub type HalfEncryptionLevel {
21   r1b: ByteArray,
22   r2_g1b: ByteArray,
23   r4b: ByteArray,
24 }
25 pub type FullEncryptionLevel {
26   r1b: ByteArray,
27   r2_g1b: ByteArray,
28   r2_g2b: ByteArray,
29   r4b: ByteArray,
30 }
```

Listing 4: The Encryption datum type

```

1 pub type EncryptionMintRedeemer {
2     EntryEncryptionMint(SchnorrProof, BindingProof)
3     LeaveEncryptionBurn(AssetName)
4 }
5 pub type EncryptionSpendRedeemer {
6     RemoveEncryption
7     // R5 witness, R5, bid token, binding proof
8     UseEncryption(ByteArray, ByteArray, AssetName, BindingProof)
9     // groth proof and public come from groth_witness withdraw redeemer
10    UseSnak
11    CancelEncryption
12    UpdateEncryptionPrice(Int)
13 }
14 pub type BindingProof {
15     z_a_b: ByteArray,
16     z_r_b: ByteArray,
17     t_1_b: ByteArray,
18     t_2_b: ByteArray,
19 }

```

Listing 5: The Encryption redeemer types

```

1 /// Example Usage:
2 ///
3 /// input:
4 /// 1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef#24
5 ///
6 /// token_name:
7 /// 181234567890abcdef1234567890abcdef1234567890abcdef1234567890abcd
8 ///
9 pub fn generate_token_name(inputs: List<Input>) -> AssetName {
10     let input: Input = builtin.head_list(inputs)
11     let id: TransactionId = input.output_reference.transaction_id
12     let idx: Int = input.output_reference.output_index
13     id |> bytearray.push(idx) |> bytearray.slice(0, 31)
14 }

```

Listing 6: Token name generation

```

1  message = "ThisIsASecretMessage."
2
3
4  # generate random data for the first encryption level
5  a0 = rng()
6  r0 = rng()
7  k0 = random_fq12(a0)
8
9  # alice as a register, sk is the secret key
10 alice = Register(sk)
11
12 # generate the r terms
13 r1b = scale(g, r0)
14 r2_g1b = scale(g, a0 + r0 \cdot sk)
15
16 a = to_int(blake2b(r1b))
17 b = to_int(blake2b(r1b + r2_g1b))
18
19 c = combine(combine(scale(H1, a), scale(H2, b)), H3)
20 r4b = scale(c, r0)
21
22 # encrypt the message
23 nonce, aad, ct = encrypt(r1, k0, message)

```

Listing 7: Creating the Encrypted data and first encryption level

```

1  pub type HalfEncryptionLevel {
2      r1b,
3      r2_g1b,
4      r4b,
5  }
6  pub type Capsule {
7      nonce,
8      aad,
9      ct: ciphertext,
10 }

```

Listing 8: Encryption data format

```

1  assert pair(g, r4b) = pair(r1b, c)

```

Listing 9: First level validation

```

1  expected_k0 = pair(r2_g1b, H0) / pair(r1b, scale(H0, sk))
2  assert k0 == expected_k0

```

Listing 10: Alice can decrypt the key

```

1 pub type EncryptionDatum {
2     owner_vkh,
3     owner_g1,
4     token: generate_token_name(inputs),
5     half_level: HalfEncryptionLevel {
6         r1b,
7         r2_g1b,
8         r4b,
9     },
10    full_level: None,
11    capsule: Capsule {
12        nonce,
13        aad,
14        ct: ciphertext,
15    },
16    status: Open,
17    new_price: 0,
18 }

```

Listing 11: Full first level datum

```

1 pub type BidDatum {
2     owner_vkh,
3     owner_g1,
4     pointer: generate_token_name(inputs),
5     token,
6     locked_until: now + 2 * minimum_bid_lock,
7     new_price: bid_price,
8 }

```

Listing 12: Full Bid datum

```

1 a1 = rng()
2 r1 = rng()
3 k1 = random_fq12(a1)
4
5 hk = to_int(k1)
6
7 r1b = scale(g, r1)
8 r2_g1b = combine(scale(g, a1), scale(bob_public_value, r1))
9
10 a = to_int(generate(r1b))
11 b = to_int(generate(r1b + r2_g1b))
12 c = combine(scale(H1, a), scale(H2, b))
13 r4b = scale(c, r1)
14
15 r5b = combine(scale(p, hk), scale(invert(H0), sk))

```

Listing 13: Generate the next level

```

1 // Bob's new half-level (current owner)
2 pub type HalfEncryptionLevel {
3     r1b,
4     r2_g1b,
5     r4b,
6 }
7 // Alice's full-level (previous owner, stored for decryption)
8 pub type FullEncryptionLevel {
9     r1b: alice.r1b,
10    r2_g1b: alice.r2_g1b,
11    r2_g2b: r5b,
12    r4b: alice.r4b,
13 }

```

Listing 14: Bob's half-level and Alice's full-level

```

1 pub type EncryptionDatum {
2     owner_vkh: bob.owner_vkh,
3     owner_g1: bob.owner_g1,
4     token,
5     half_level: HalfEncryptionLevel {
6         r1b,
7         r2_g1b,
8         r4b,
9     },
10    full_level: Some(FullEncryptionLevel {
11        r1b: alice.r1b,
12        r2_g1b: alice.r2_g1b,
13        r2_g2b: r5b,
14        r4b: alice.r4b,
15    }),
16    capsule: Capsule {
17        nonce,
18        aad,
19        ct: ciphertext,
20    },
21    status: Open,
22    new_price: bid.new_price,
23 }

```

Listing 15: Bob's encryption datum

```

1 assert pair(g, bob.r4b) = pair(bob.r1b, c)
2 assert pair(g, alice.r5b) \cdot pair(alice.u, H0) = pair(alice.witness, p)

```

Listing 16: Validate the re-encryption process

```

1
2 # Reconstruct encryption_levels by querying blockchain history
3 encryption_levels = query_chain_history(token, from_mint=True)
4
5 h0x = scale(H0, sk)
6 shared = h0x
7
8 for entry in encryption_levels:
9     r1 = entry.r1b
10
11     if is_half_level(entry):
12         r2 = pair(entry.r2_g1b, H0)
13     else:
14         r2 = pair(entry.r2_g1b, H0) \cdot pair(r1, entry.r2_g2b)
15
16     b = pair(r1, shared)
17     key = mimc_hash(r2 / b, DOMAIN_TAG)
18     k = to_int(key)
19     shared = scale(p, k)
20
21 message = decrypt(r1, key, capsule.nonce, capsule.ct, capsule.aad)

```

Listing 17: Decrypting the secret message

```

1 pub fn fiat_shamir_heuristic(
2     // compressed g element
3     g_b: ByteArray,
4     // compressed g^r element
5     g_r_b: ByteArray,
6     // compressed g^x element
7     u_b: ByteArray,
8 ) -> ByteArray {
9     // concat g_b, g_r_b, u_b, and b together then hash the result
10    schnorr_domain_tag
11        |> bytearray.concat(g_b)
12        |> bytearray.concat(g_r_b)
13        |> bytearray.concat(u_b)
14        |> crypto.blake2b_224()
15 }

```

Listing 18: Fiat Shamir Transform Heuristic

B Appendix B - Proofs

Lemma B.1. *Correctness for Algorithm 1, a non-interactive Schnorr's Σ -protocol for the discrete logarithm relation.*

Proof. We start with (g, u, a, z) where $g \in \mathbb{G}_1$, $u = [\delta]g \in \mathbb{G}_1$, $a = [r]g \in \mathbb{G}_1$, and $z = r + c \cdot \delta \in \mathbb{Z}_n$.

Use the Fiat-Shamir transform to generate a challenge value $c = R(g, u, a)$.

$$[z]g = a + [c]u$$

$$[z]g = [r]g + [c][x]g$$

$$[z]g = [r + c \cdot x]g$$

An honest **Register** can produce an a and z that will satisfy $[z]g = a + [c]u$ proving knowledge of the secret δ .

□

Lemma B.2. *Correctness for Algorithm 2, a non-interactive Σ -protocol for binding user data to encryption data.*

Proof. We start with $(g, u, t_1, t_2, z_a, z_r, r_1, \chi)$ where $g \in \mathbb{G}_1$, $u = [\delta]g \in \mathbb{G}_1$, $t_1 = [\rho]g \in \mathbb{G}_1$, $t_2 = [\alpha]g + [\rho]u \in \mathbb{G}_1$, $z_a = \alpha + c \cdot a \in \mathbb{Z}_n$, $z_r = \rho + c \cdot r \in \mathbb{Z}_n$, $r_1 = [r]g \in \mathbb{G}_1$, and $\chi = [a + r \cdot \delta]g \in \mathbb{G}_1$.

Use the Fiat-Shamir transform to generate a challenge value $c = R(g, u, t_1, t_2)$.

$$[z_a]g + [z_r]u = t_2 + [c]\chi \wedge [z_r]g = t_1 + [c]r_1$$

$$[z_a]g + [z_r][\delta]g = [\alpha]g + [\rho][\delta]g + [c][a + r \cdot \delta]g \wedge [z_r]g = [\rho]g + [c][r]g$$

$$[z_a + z_r \cdot \delta]g = [\alpha + \rho \cdot \delta + c \cdot a + c \cdot r \cdot \delta]g \wedge [z_r]g = [\rho + c \cdot r]g$$

$$[z_a + z_r \cdot \delta]g = [\alpha + c \cdot a + \rho \cdot \delta + c \cdot r \cdot \delta]g \wedge [z_r]g = [\rho + c \cdot r]g$$

An honest **Register** can produce an t_1 , t_2 , z_a , and z_r that will satisfy $z_a + z_r \cdot \delta = \alpha + c \cdot a + \rho \cdot \delta + c \cdot r \cdot \delta$ and $z_r = \rho + c \cdot r$ proving knowledge of the secret a and r while using u .

□

Lemma B.3. *Correctness for Algorithm 6, recursive decryption*

Proof. Lets assume we have a single full encryption level and a half encryption level given by $(r_{1,a}, r_{2,a})$ and $(r_{1,b}, r_{2,b})$, respectively, where $r_{1,i} \in \mathbb{G}_1$, and $r_{2,i} \in \mathbb{G}_T$. The owner of the half level has secret $y \in \mathbb{Z}_n$ and the owner of the full level has secret $x \in \mathbb{Z}_n$. The point $h_0 = [s_0]p \in \mathbb{G}_2$ is a fixed public element.

Calculate κ_1 from the half level.

$$\kappa_1 = \frac{r_{2,b}}{e(r_{1,b}, h_0^y)} = \frac{e(q^{a_1+y \cdot r_1}, h_0)}{e(q^{r_1}, h_0^y)} = e(q^{a_1+y \cdot r_1}, h_0) \cdot e(q^{r_1}, h_0^y)^{-1}$$

$$\kappa_1 = e(q^{a_1+y \cdot r_1}, h_0) \cdot e(q^{-r_1 \cdot y}, h_0)$$

$$\kappa_1 = e(q^{a_1}, h_0)$$

Calculate κ_0 from the full level.

$$\kappa_0 = \frac{r_{2,a}}{e(r_{1,a}, p^{H(\kappa_1)})} = \frac{e(q^{a_0+x \cdot r_0}, h_0) \cdot e(q^{r_0}, p^{H(\kappa_1)} \cdot h_0^{-x})}{e(q^{r_0}, p^{H(\kappa_1)})}$$

$$\kappa_0 = e(q^{a_0+x \cdot r_0}, h_0) \cdot e(q^{r_0}, p^{H(\kappa_1)} \cdot h_0^x) \cdot e(q^{-r_0 \cdot H(\kappa_1)}, p)$$

$$\kappa_0 = e(q^{s \cdot a_0 + s \cdot x \cdot r_0}, p) \cdot e(q^{r_0 \cdot H(\kappa_1) - r_0 \cdot s \cdot x}, p) \cdot e(q^{-r_0 \cdot H(\kappa_1)}, p)$$

$$\kappa_0 = e(q^{s \cdot a_0}, p) = e(q^{a_0}, h_0)$$

Any user with κ_0 may decrypt the original message. When many full levels exist then the recursive relationship holds in general.

$$\kappa_{i-1} = \frac{r_{2,i-1}}{e(r_{1,i-1}, p^{H(\kappa_i)})} = e(q^{a_{i-1}}, h_0)$$

□

Bibliography

- [1] J. Stone, “Stuff.io whitepaper 1.0.” Accessed: Oct. 25, 2025. [Online]. Available: <https://book-io.medium.com/stuff-io-whitepaper-1-0-9529db7cdeaf>
- [2] M. Mambo and E. Okamoto, “Proxy cryptosystems: Delegation of the power to decrypt ciphertexts,” *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E80–A, no. 1, pp. 54–63, 1997, Available: https://search.ieice.org/bin/summary.php?id=e80-a_1_54
- [3] M. Blaze, G. Bleumer, and M. Strauss, “Divertible protocols and atomic proxy cryptography,” in *EUROCRYPT 1998*, Springer, 1998. doi: [10.1007/BFb0054122](https://doi.org/10.1007/BFb0054122).
- [4] G. Ateniese, K. Fu, M. Green, and S. Hohenberger, “Improved proxy re-encryption schemes with applications to secure distributed storage,” in *NDSS 2005*, 2005. Available: <https://www.ndss-symposium.org/ndss2005/improved-proxy-re-encryption-schemes-applications-secure-distributed-storage/>
- [5] IronCore Labs, *Recrypt (rust): Transform/proxy re-encryption library*. Accessed: Oct. 25, 2025. [Online]. Available: <https://github.com/IronCoreLabs/recrypt-rs>
- [6] H. Wang and Z. Cao, “A fully secure unidirectional and multi-use proxy re-encryption scheme.” Poster, 16th ACM Conference on Computer and Communications Security (CCS 2009), Chicago, IL, USA, Nov. 9–13, 2009, 2009. Available: https://www.sigsac.org/ccs/CS2009/pd/abstract_16.pdf
- [7] *IEEE standard specifications for public-key cryptography-amendment 1: Additional techniques*. IEEE, 2004.
- [8] *Advanced encryption standard (AES)*. NIST, 2001.
- [9] S. Bowe, “BLS12-381: New zk-SNARK elliptic curve construction.” Accessed: Oct. 25, 2025. [Online]. Available: <https://electriccoin.co/blog/new-snark-curve/>
- [10] aiken-lang contributors, *Aiken: A modern smart contract platform for cardano*. (2025). Accessed: Dec. 16, 2025. [Online]. Available: <https://github.com/aiken-lang/aiken>
- [11] N. El Mrabet and M. Joye, Eds., *Guide to pairing-based cryptography*. in Chapman & hall/CRC cryptography and network security. New York: Chapman; Hall/CRC, 2017, p. 420. doi: [10.1201/9781315370170](https://doi.org/10.1201/9781315370170).
- [12] S. Josefsson and I. Liusvaara, *Edwards-curve digital signature algorithm (EdDSA)*. IETF, 2017. Available: <https://www.rfc-editor.org/rfc/rfc8032>
- [13] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Advances in cryptology – CRYPTO ’86*, in Lecture notes in computer science, vol. 263. Springer, 1986, pp. 186–194. doi: [10.1007/3-540-47721-7_12](https://doi.org/10.1007/3-540-47721-7_12).
- [14] J.-P. Aumasson, S. Neves, Z. Wilcox-O’Hearn, and C. Winnerlein, *The BLAKE2 cryptographic hash and message authentication code (MAC)*. IETF, 2015. Available: <https://www.rfc-editor.org/rfc/rfc7693>
- [15] D. Hopwood, S. Bowe, T. Hornby, and N. Wilcox, “Zcash protocol specification,” Electric Coin Company, Version 2022.3.8 [NU5], Sep. 2022. Available: <https://resources.cryptocompare.com/asset-management/102/1672746915982.pdf>
- [16] J. Thaler, “Proofs, arguments, and zero-knowledge,” *Foundations and Trends in Privacy and Security*, vol. 4, no. 2–4, pp. 117–660, 2022, doi: [10.1561/33000000030](https://doi.org/10.1561/33000000030).

-
- [17] C.-P. Schnorr, “Efficient signature generation by smart cards,” *Journal of Cryptology*, vol. 4, pp. 161–174, 1991, doi: [10.1007/BF00196725](https://doi.org/10.1007/BF00196725).
 - [18] A. J. Menezes, *Elliptic curve public key cryptosystems*, vol. 234. in The springer international series in engineering and computer science, vol. 234. Boston, MA: Kluwer Academic Publishers, 1993. doi: [10.1007/978-1-4615-3198-2](https://doi.org/10.1007/978-1-4615-3198-2).
 - [19] H. Krawczyk, “Cryptographic extraction and key derivation: The HKDF scheme.” Cryptology ePrint Archive, Paper 2010/264, 2010. Available: <https://eprint.iacr.org/2010/264>
 - [20] A. Konrad and T. Vellekoop, “CIP-68: Datum metadata standard.” <https://cips.cardano.org/cip/CIP-68>, 2022.